

Unidad 3

Procesos y redirecciones de entrada/salida

Jesús Torres

Vamos a extender el servidor de backup desarrollado en la Unidad 2 para que pueda comprimir los archivos durante la copia, aplicando conocimientos de creación de procesos, tuberías anónimas y redirecciones de entrada/salida. Aprenderemos a usar `fork()`, `exec()`, `pipe()` y `dup2()` para ejecutar comandos externos y comunicarnos con ellos.

Contenidos

1. Actividad 3	3
1.1. Objetivo	3
1.2. Modificaciones al sistema	3
1.3. Línea de Comandos	3
1.3.1. Ejemplos de ejecución	5
1.4. Arquitectura de compresión	5
1.5. Implementación	6
1.5.1. Procesamiento de argumentos de línea de comandos	6
1.5.2. Obtener el comando de compresión	9
1.5.3. Verificar disponibilidad de los comandos	9
1.5.4. Extensiones de archivo	10
1.5.5. Función <code>copy_file_compressed()</code>	10
1.5.6. Modificación del bucle principal	13
1.6. Manejo de Errores	13
1.7. Comprobación	14
1.7.1. Preparación del entorno	14
1.7.2. Prueba sin compresión (regresión)	14
1.7.3. Prueba con compresión gzip	15
1.7.4. Casos de error	15
A. Procesamiento de línea de comandos	17
A.1. Argumentos de la línea de comandos	17
A.2. Función <code>parse_arguments()</code>	18
A.3. Procesamiento con <code>getopt()</code>	21
A.3.1. Variables globales de <code>getopt()</code>	22

A.3.2. Cadena de opciones <code>optstring</code>	22
A.3.3. Valor de retorno	22
A.3.4. Uso básico de <code>getopt()</code>	23
A.3.5. Implementación de completa de <code>parse_arguments()</code> con <code>getopt()</code> . .	23
B. Gestión de procesos	26
B.1. Creación de procesos con <code>fork()</code>	26
B.1.1. Descriptores de archivo compartidos	27
B.2. Reemplazo de programa con <code>exec()</code>	28
B.2.1. Diferencias entre las variantes de <code>exec()</code>	28
B.2.2. Ejemplo con <code>execvp()</code>	30
B.3. Espera de procesos hijos con <code>wait()</code>	31
B.3.1. Interpretar el estado de salida	32
B.3.2. Ejemplo con comando externo	33
B.3.3. Importancia de <code>wait()</code>	34
B.4. Redirección de entrada/salida	34
B.4.1. Ejemplos de redirección a archivos	34
B.4.2. Redirección con <code>fork()</code> y <code>exec()</code>	35
C. Tuberías anónimas	37
C.1. Creación de tuberías anónimas	38
C.1.1. Uso básico de tuberías anónimas	38
C.2. Comunicación entre procesos con tuberías	40
C.3. Redirección con tuberías	42
C.3.1. Redirigir <code>stdout</code> a una tubería	42
C.3.2. Redirigir <code>stdin</code> desde una tubería	43
C.4. Patrón completo: Pipe + Fork + Exec + Redirección	45
C.5. Redirección bidireccional	46

1. Actividad 3

1.1. Objetivo

Vamos a extender el servidor de backup desarrollado en la Unidad 2 para que pueda comprimir los archivos durante la copia, aplicando los conocimientos de creación de procesos, tuberías anónimas y redirecciones de entrada/salida.

El servidor debe:

- Aceptar opciones de línea de comandos para especificar el tipo de compresión deseado.
- Si se indican alguna de estas opciones, crear procesos hijos que ejecuten comandos de compresión (**gzip**, **bzip2**, **xz**).
- Usar tuberías anónimas para comunicar el proceso padre con el proceso hijo con el comando de compresión.
- Redirigir la entrada y salida estándar del proceso hijo para conectarlo con el padre a través de una tubería y con el archivo de destino.
- Añadir la extensión apropiada al archivo comprimido según el formato de compresión.

1.2. Modificaciones al sistema

La arquitectura del sistema permanece igual que en la Unidad 2:

- El cliente **backup** envía solicitudes al servidor mediante FIFO y señales (sin modificaciones).
- El servidor **backup-server** procesa las solicitudes y copia los archivos.

Como se muestra en la Figura 1, la única diferencia es que ahora el servidor puede **comprimir** los archivos durante la copia si se especifica una opción de compresión en la línea de comandos.

1.3. Línea de Comandos

El programa **backup-server** ahora aceptará opciones adicionales:

```
backup-server [-z | -j | -x] [DIRECTORIO_DESTINO]
```

Donde:

- z (opcional)** Comprime los archivos usando **gzip** (extensión **.gz**).
- j (opcional)** Comprime los archivos usando **bzip2** (extensión **.bz2**).
- x (opcional)** Comprime los archivos usando **xz** (extensión **.xz**).
- DIRECTORIO_DESTINO (opcional)** Ruta del directorio donde se guardarán las copias de seguridad. Si no se indica, se utilizará el directorio actual de trabajo.

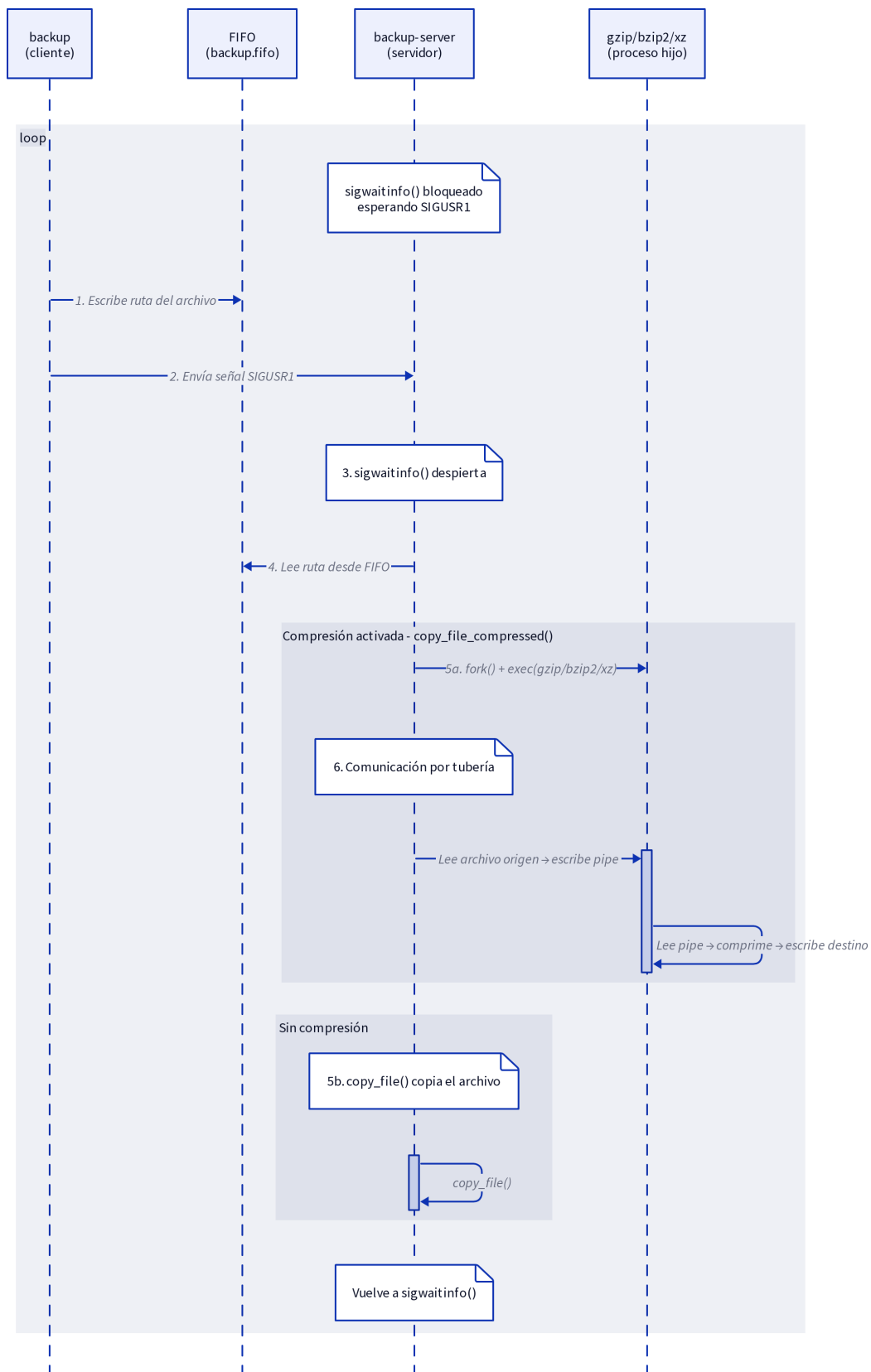


Figura 1: Flujo de comunicación entre cliente y servidor de backup

Las opciones de compresión son **mutuamente excluyentes**. Es decir, solo se puede especificar una a la vez e indicar más de una debe producir un error.

1.3.1. Ejemplos de ejecución

Estos son algunos ejemplos de uso correcto del programa `backup-server` con las nuevas opciones de compresión:

```
# Sin compresión (comportamiento de la Unidad 2)
$ backup-server /tmp/backups
backup-server: esperando solicitudes de backup en /tmp/backups

# Con compresión gzip
$ backup-server -z /tmp/backups
backup-server: compresión: gzip
backup-server: esperando solicitudes de backup en /tmp/backups

# Con compresión xz y backup en el directorio actual
$ cd /tmp/backups
$ backup-server -x
backup-server: compresión: xz
backup-server: esperando solicitudes de backup en /tmp/backups
```

Mientras que estos son algunos ejemplos de uso incorrecto que deben producir mensajes de error:

```
# Múltiples opciones de compresión (ERROR)
$ backup-server -z -j /tmp/backups
backup-server: error: solo se admite una opción de compresión
uso: backup-server [-z | -j | -x] [DIRECTORIO_DESTINO]

# Opción inválida
$ backup-server -w /tmp/backups
backup-server: error: opción desconocida
uso: backup-server [-z | -j | -x] [DIRECTORIO_DESTINO]
```

1.4. Arquitectura de compresión

Cuando el servidor es ejecutado con una opción de compresión, en lugar de copiar el archivo directamente al destino usando `copy_file()`, empleará una nueva función llamada `copy_file_compressed()` que debe crear un proceso hijo que ejecute el comando de compresión adecuado (`gzip`, `bzip2` o `xz`).

Para ello, `copy_file_compressed()` utiliza un **proceso hijo** que ejecuta el comando de compresión apropiado. La comunicación entre el proceso padre (servidor) y el proceso hijo (comando de compresión) para enviar los datos que deben ser comprimidos y guardados se realiza mediante una **tubería anónima**.

Como se ilustra en la Figura 2, el sistema funciona de la siguiente manera:

1. El padre crea una tubería anónima con `pipe()` (ver la Sección C.1).
2. El padre crea un proceso hijo con `fork()` (ver la Sección B.1).
3. **Proceso hijo:**
 - Redirige `stdin` (descriptor 0) al extremo de lectura de la tubería (`pipe[0]`) usando `dup2()` (ver la Sección B.4).
 - Abre el archivo de destino para escritura con `open()` como se indicó en la Unidad 1. Deben usarse las opciones adecuadas de `open()` para que el archivo sea creado si no existe y truncado si existe.
 - Redirige `stdout` (descriptor 1) al archivo de destino usando `dup2()`.
 - Cierra los descriptores originales que ya no son necesarios.
 - Ejecuta el comando de compresión con `execvp()` (ver la Sección B.2).
4. **Proceso padre:**
 - Bloquea la señal `SIGPIPE`.
 - Cierra los descriptores de archivo de la tubería que no son necesarios.
 - Lee del archivo origen en bloques de 64 KiB.
 - Escribe cada bloque en el extremo de escritura de la tubería (`pipe[1]`).
 - Cuando termina, cierra `pipe[1]` para señalar EOF al hijo.
 - Espera a que el hijo termine con `wait()` o `waitpid()` (ver la Sección B.3).
 - Verifica el código de salida del hijo para detectar errores.

! Importante

El proceso padre tiene que bloquear la señal `SIGPIPE` durante la ejecución de `copy_file_compressed()` para evitar que el servidor termine inesperadamente si el proceso hijo termina y el padre intenta escribir en la tubería (ver en “Escribir en una tubería rota” en el Apéndice A en la Unidad 2).

1.5. Implementación

1.5.1. Procesamiento de argumentos de línea de comandos

Para procesar las opciones y argumentos de la línea de comandos se implementará la función `parse_arguments()`:

```
std::expected<ServerOptions, ParseArgsErrors>
parse_arguments(int argc, char* argv[]);
```

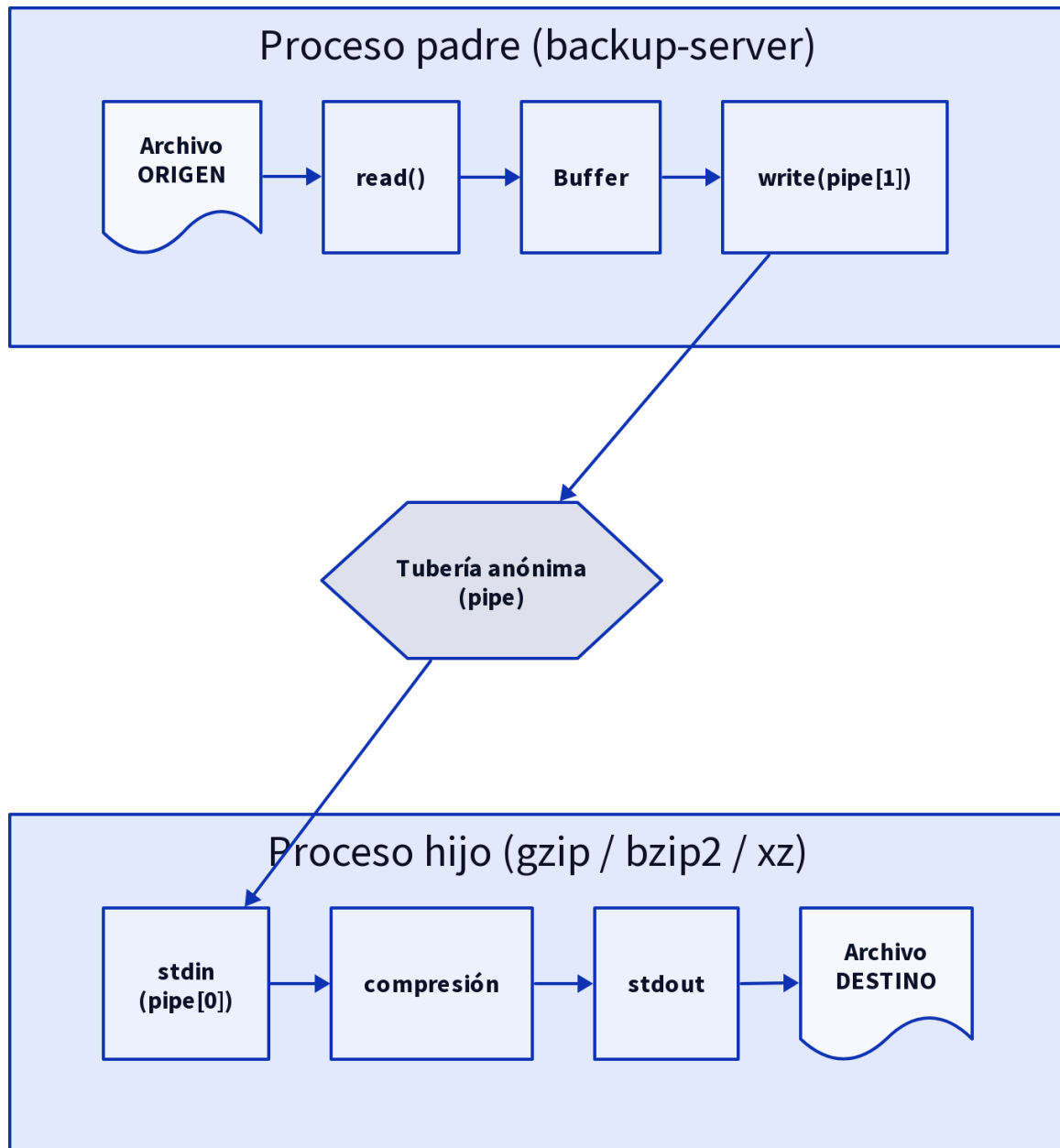


Figura 2: Arquitectura de compresión usando un proceso hijo y una tubería anónima

Esta función debe:

- Analizar los argumentos de línea de comandos usando las técnicas descritas en el Apéndice A.1.
- Verificar que solo se especifica una opción de compresión.
- Extraer el directorio de destino si se proporciona como argumento.
- Retornar un objeto `ServerOptions` con la configuración del servidor o un código de error `ParseArgsErrors`.

```
struct ServerOptions
{
    enum class CompressionType
    {
        NONE,
        GZIP,
        BZIP2,
        XZ
    };

    CompressionType compression = CompressionType::NONE;
    std::string backup_dir;
};
```

```
enum class ParseArgsErrors
{
    unknown_option,
    multiple_compression_options,
    too_many_arguments,
    //
    // Aquí otros errores posibles de parse_arguments()...
    //
};
```

La función se usará al inicio del programa para procesar los argumentos. Por ejemplo:

```
int main(int argc, char* argv[])
{
    auto arguments_result = parse_arguments(argc, argv);

    if (!arguments_result.has_value())
    {
        switch (arguments_result.error())
        {
            case ParseArgsErrors::unknown_option:
                std::println(stderr,
```



```
        "backup-server: error: opción desconocida");
        break;

    // Manejo del resto de posibles condiciones de error...
}

print_usage();
return EXIT_FAILURE;
}

ServerOptions options = arguments_result.value();

// Continuar con la inicialización del servidor...
}
```

1.5.2. Obtener el comando de compresión

Para facilitar la obtención del comando de compresión adecuado según la opción seleccionada, se implementará la siguiente función:

```
std::string get_compression_command(
    ServerOptions::CompressionType compression);
```

Simplemente debe devolver el nombre del comando correspondiente a cada posible valor de `ServerOptions::CompressionType`.

1.5.3. Verificar disponibilidad de los comandos

Durante la inicialización, el servidor debe verificar que el comando de compresión solicitado está instalado en el sistema y mostrar un error en caso contrario. Para ello se implementará la siguiente función:

```
bool is_command_available(const std::string& command);
```

Esta función debe:

- Leer la variable de entorno `PATH` y extraer las rutas que contiene (recuerda que se trata de una lista de directorios separadas por `'.'`).
- Para cada ruta, construir la posible ruta absoluta del comando concatenándola con `/ + command`. Por ejemplo, si en `PATH` está el directorio `/usr/bin` y el comando es `gzip` la posible ruta será `/usr/bin/gzip`.
- Comprobar si el comando existe en la ruta calculada usando `access()` con el flag `X_OK`.

Si se encuentra el comando en alguna de las rutas de PATH el comando devolverá `true`. En otro caso, devuelve `false`.

Comprobado la opción `ServerOptions::compression` se puede llamar a `is_command_available` con el nombre del comando correspondiente para verificar su disponibilidad:

```
if (config.compression == ServerOptions::CompressionType::GZIP)
{
    if (!is_command_available("gzip"))
    {
        std::println(stderr,
            "backup-server: error: gzip no está instalado");
        return EXIT_FAILURE;
    }
}
```

1.5.4. Extensiones de archivo

El servidor debe añadir la extensión apropiada al nombre del archivo de destino según el formato de compresión. Para eso, se propone crear una función que devuelva una cadena con la extensión adecuada para cada formato de compresión posible:

```
std::string get_compression_extension(
    ServerOptions::CompressionType compression);
```

Esta función retorna:

- `".gz"` para `ServerOptions::CompressionType::GZIP`
- `".bz2"` para `ServerOptions::CompressionType::BZIP2`
- `".xz"` para `ServerOptions::CompressionType::XZ`
- `""` (cadena vacía) para `ServerOptions::CompressionType::NONE`

! Importante

El valor devuelto por esta función se debe concatenar al final de la ruta del archivo de destino, antes de llamar a `copy_file_compressed()` para hacer la copia.

1.5.5. Función `copy_file_compressed()`

Esta es la función principal que realiza la copia con compresión:

```
std::expected<void, CopyFileCompressedError> copy_file_compressed(
    const std::string& src_path,
    const std::string& dest_path,
    const std::string& compression_command);
```

Donde:

src_path Ruta del archivo de origen.

dest_path Ruta del archivo de destino (ya con la extensión apropiada).

compression_command Nombre del comando de compresión ("gzip", "bzip2", o "xz"), según la opción seleccionada en la línea de comandos.

La función debe implementar la arquitectura descrita en Sección 1.4 usando `pipe()`, `fork()`, `dup2()`, `execvp()`, `wait()` y las funciones de lectura/escritura de archivos vistas en la Unidad 1. El uso detallado de `fork()`, `exec()`, `wait()` y `dup2()` se explica en el Apéndice B. El uso de `pipe()` y la comunicación entre procesos mediante tuberías se explica en el Apéndice C.

1.5.5.1. Aspectos importantes

Respecto a la implementación de `copy_file_compressed()`, es importante no olvidar los siguientes **aspectos críticos**:

1. **Cierre de descriptores:** Cada proceso debe cerrar los extremos de la tubería que no usa.
 - Hijo cierra `pipe[1]` (escritura).
 - Padre cierra `pipe[0]` (lectura).
2. **EOF en tubería:** Cuando el padre cierra `pipe[1]`, porque ha terminado de escribir el contenido del archivo de origen, el hijo recibe un fin de archivo (EOF) al leer de `stdin`. Es decir, su `read()` de `stdin` retornará 0 cuando haya leído todo el contenido de la tubería.
3. **Manejo de errores en el hijo:** Si el proceso hijo encuentra errores, debe terminar con `std::exit()`. Respecto al código de salida al terminar el hijo en este caso:
 - Si `open()` no puede abrir el archivo de destino, usar el código 128.
 - Si `exec()` falla con `ENOENT` (archivo no encontrado), usar el código 127.
 - Si `exec()` falla con `EACCES` (permiso denegado) o `ENOEXEC` (formato ejecutable incorrecto), usar el código 126.
 - Para otros errores, usar el código 255.
4. **Espera del hijo:** El padre debe esperar con `wait()` o `waitpid()` para:
 - Evitar procesos zombies.
 - Verificar el código de salida del hijo, detectar errores y comunicarlo al llamador de `copy_file_compressed()`.
5. **Redirecciones:** Usar `dup2()` para redirigir descriptores y luego cerrar los descriptores originales.

6. **Bloqueo de señales:** Al menos durante la ejecución de `copy_file_compressed()`, el proceso padre debe bloquear la señal `SIGPIPE` para evitar que el servidor termine inesperadamente si el proceso hijo termina y el padre intenta escribir en la tubería cerrada que los conectaba (ver en “Escribir en una tubería rota” en el Apéndice A en la Unidad 2).

1.5.5.2. Manejo de errores

Observa, que la función devuelve como tipo de error un nuevo *enum* llamado `CopyFileCompressedError` que debe definir los posibles errores que pueden ocurrir durante este modo de copia. El motivo de usar un *enum* específico es que algunas condiciones de error de `copy_file_compressed()` no se mapean fácilmente en los valores posibles de `errno`, por lo que no sirve usar `std::system_error` como con `copy_file()`¹².

El *enum* `CopyFileCompressedError` podría definirse de la siguiente manera:

```
enum class CopyFileCompressedError
{
    command_access_denied,
    command_execution_failed,
    command_not_found,
    output_access_denied,
    process_creation_failed,
    pipe_creation_failed,
    unknown_error
};
```

Cada uno de estos códigos de error corresponde a una de las siguientes condiciones:

- pipe_creation_failed** Cualquier error al crear la tubería con `pipe()`.
- process_creation_failed** Error al crear el proceso hijo con `fork()`.
- output_access_denied** El proceso hijo termina con código de salida 128, que corresponde a que `open` falló al abrir el archivo de destino.
- command_not_found** El proceso hijo termina con código de salida 127, que corresponde a que `exec()` falló con `ENOENT`.
- command_access_denied** El proceso hijo termina con código de salida 126, que corresponde a que `exec()` falló con `EACCES` o `ENOEXEC`.
- command_execution_failed** El proceso hijo termina con un código de salida distinto de 0, 126 o 127, indicando que el comando de compresión falló durante su ejecución o que ocurrió cualquier otro problema en el proceso hijo que impidió ejecutar el comando.
- unknown_error** Cualquier otra condición de error no contemplada en los casos anteriores.

¹En particular, no se puede representar con valores de `errno` errores como que el comando de compresión termine inesperadamente con un código de error o sea interrumpido por una señal.

²Esto no es del todo exacto. C++ permite definir códigos y categorías de error personalizadas, según las necesidades de nuestras librerías y programas, que pueden integrarse con `std::system_error`, pero eso se sale del alcance de esta práctica.

1.5.6. Modificación del bucle principal

El bucle principal del servidor debe modificarse para introducir la decisión de si usar `copy_file()` o `copy_file_compressed()`, como se muestra en el siguiente pseudocódigo:

```
while True:
    # Código anterior desarrollado en la Unidad 2

    nombre_archivo = get_filename(ruta_origen)
    ruta_destino = directorio_backups + "/" + nombre_archivo

    if config.compression == NONE: ①
        resultado = copy_file(ruta_origen, ruta_destino)
    else: ②
        extension = get_compression_extension(config.compression)
        ruta_destino = ruta_destino + extension
        comando = get_compression_command(config.compression)
        resultado = copy_file_compressed(ruta_origen, ruta_destino, comando)

    if has_error(resultado):
        print(stderr, "error al hacer backup de {ruta_origen}: {error}")
    else:
        print("backup completado: {ruta_origen} -> {ruta_destino}")
```

- ① Si no hay compresión, se llama a `copy_file()` como antes.
- ② Si la compresión está activada, se añade la extensión a la ruta de destino y se llama a `copy_file_compressed()` con el comando de compresión adecuado.

1.6. Manejo de Errores

El servidor debe manejar errores adicionales relacionados con la compresión, siguiendo siempre las pautas de manejo de errores descritas en la Unidad 1.

▪ Durante la inicialización:

-  El comando de compresión no está instalado.
-  Múltiples opciones de compresión especificadas.
-  Opciones de línea de comandos inválidas.

▪ Durante la compresión:

-  Error al crear la tubería (`pipe()` falla).
-  Error al crear el proceso hijo (`fork()` falla).
- Error al ejecutar el comando de compresión (`exec()` falla en el hijo).
- El comando de compresión termina con código de error.
- El comando de compresión es terminado por una señal.
- Error al escribir en la tubería (tubería rota, `SIGPIPE`).

En todos los casos se deben **mostrar mensajes de error descriptivos por la salida de error**.

Los casos marcados con **⚠** debe provocar la terminación inmediata de la ejecución del servidor, saliendo por `main()` con un código de error distinto de `EXIT_SUCCESS`, como se explicó en la Unidad 1. Recuerda que **antes de salir es necesario liberar todos los recursos** (descriptores, memoria, terminar y esperar por procesos hijos en ejecución, etc.).

En el resto de situaciones de error, el servidor debe continuar ejecutándose y procesar las siguientes solicitudes.

1.7. Comprobación

1.7.1. Preparación del entorno

Para probar el servidor con compresión, primero debemos preparar el entorno de trabajo creando los directorios necesarios:

```
$ export BACKUP_WORK_DIR=/tmp/backup-work
$ mkdir -p /tmp/backup-work
$ mkdir -p /tmp/backups
```

Y algunos archivos de prueba:

```
$ echo "Archivo de texto largo para comprimir..." > /tmp/test.txt
$ dd if=/dev/urandom of=/tmp/binary.dat bs=1M count=10 iflag=fullblock
```

1.7.2. Prueba sin compresión (regresión)

Primero conviene verificar que el servidor funciona correctamente sin compresión, para asegurarnos de que las modificaciones no han roto la funcionalidad original implementada en la Unidad 2.

1. Iniciar el servidor sin opciones de compresión.

```
$ backup-server /tmp/backups
backup-server: esperando solicitudes de backup en /tmp/backups
```

2. En otra terminal, solicitar un backup.

```
$ backup /tmp/test.txt
backup: solicitud enviada para /tmp/test.txt
```

3. Finalmente, verificar que el archivo se ha copiado correctamente.

```
$ ls -lh /tmp/backups/  
-rw-r--r-- 1 user user 51 nov  5 10:00 test.txt  
  
$ cmp /tmp/test.txt /tmp/backups/test.txt
```

1.7.3. Prueba con compresión gzip

De forma similar, podemos probar el servidor con compresión gzip:

```
$ backup-server -z /tmp/backups  
backup-server: compresión: gzip  
backup-server: esperando solicitudes de backup en /tmp/backups
```

```
$ backup /tmp/test.txt  
backup: solicitud enviada para /tmp/test.txt
```

Entonces, en la terminal del servidor deberíamos ver:

```
backup-server: backup completado: /tmp/test.txt -> /tmp/backups/test.txt.gz
```

Posteriormente, podemos verificar que el archivo comprimido se ha creado correctamente:

```
$ ls -lh /tmp/backups/  
-rw-r--r-- 1 user user 75 nov  5 10:05 test.txt.gz  
  
$ file /tmp/backups/test.txt.gz  
/tmp/backups/test.txt.gz: gzip compressed data
```

Y finalmente, descomprimir y verificar el contenido:

```
$ gunzip -c /tmp/backups/test.txt.gz > /tmp/test_decompressed.txt  
$ cmp /tmp/test.txt /tmp/test_decompressed.txt
```

El mismo procedimiento se puede seguir para probar las opciones de compresión con **bzip2** y **xz** y para verificar que los archivos de gran tamaño, como **binary.dat**, se comprimen correctamente.

1.7.4. Casos de error

- **Comando no instalado:** Primero tendremos que desinstalar temporalmente el comando de compresión que queremos probar (por ejemplo, **bzip2**) y luego ejecutar el servidor con la opción correspondiente:

```
# Desinstalar bzip2 temporalmente
$ sudo apt remove bzip2

$ backup-server -j /tmp/backups
backup-server: error: bzip2 no está instalado
```

■ **Múltiples opciones:**

```
$ backup-server -z -j /tmp/backups
backup-server: error: solo se admite una opción de compresión
uso: backup-server [-z | -j | -x] [DIRECTORIO_DESTINO]
```


A. Procesamiento de línea de comandos

A.1. Argumentos de la línea de comandos

Como ya sabemos, los argumentos de la línea de comandos se reciben en `main()` mediante `argv`, un array de punteros a `char`:

```
int main(int argc, char* argv[])
{
    // ...
}
```

Para facilitar su procesamiento, `argv` se puede convertir en un vector de `std::string_view`:

```
int main(int argc, char* argv[])
{
    std::vector<std::string_view> args(argv + 1, argv + argc);

    // ...
}
```

Usamos `std::string_view` porque nos ofrece una funcionalidad similar a la de `std::string` pero sin hacer una copia de cada cadena. Es decir, los argumentos de la línea de comandos están en `argv`, pero accedemos a ellos a través de los objetos `std::string_view` en `args`.

Con este vector, es muy sencillo iterar para procesar los argumentos de la línea de comandos uno tras otro:

```
for (auto it = args.begin(), end = args.end(); it != end; ++it)
{
    if (*it == "-h" || *it == "--help") ①
    {
        show_help = true;
    }
    else if (*it == "-o" || *it == "--output") ②
    {
        if (++it != end) ③
        {
            output_filename = *it;
        }
        else
        {
            // Error por falta de argumento...
        }
    }
}
```

```

    else if (/* otra opción */)
    {
        // ...
    }
    else if (!it->starts_with("-")) ④
    {
        additional_args.push_back(*it); ⑤
    }

    // ...
}

```

- ① Comprobar si el argumento actual es `-h` o `--help` para activar la opción de mostrar la ayuda.
- ② Comprobar si el argumento actual es `-o` o `--output`, por ejemplo, para indicar el nombre del archivo de salida.
- ③ Avanzar el iterador para obtener el argumento de la opción `-o/--output` y guardarlo en `output_filename`.
- ④ Si el argumento actual no empieza por `-`, se considera un argumento adicional (no una opción).
- ⑤ Guardar el argumento adicional en un vector para procesarlo más adelante.

i Nota

A primera vista, podría parecer más sencillo usar un índice entero para recorrer el array `argv` en lugar de un iterador. Sin embargo, usar iteradores nos permite avanzar fácilmente con `++it` cuando una opción tiene un argumento asociado (como en el caso de `-o/--output`).

A.2. Función `parse_arguments()`

Para separar mejor la responsabilidad, se puede meter el código anterior en una función `parse_arguments()` que procese los argumentos de la línea de comandos y devuelva una estructura con los valores de las opciones encontradas:

```

enum class ParseArgsError ①
{
    missing_argument,
    unknown_option,
    // ...
};

struct ProgramOptions ②
{

```

```

    bool show_help = false;
    std::string output_filename;
    // ...
    std::vector<std::string> additional_args;
};

```

③

```

std::expected<ProgramOptions, ParseArgsError>
parse_arguments(int argc, char* argv[])
{
    std::vector<std::string_view> args(argv + 1, argv + argc);
    ProgramOptions options;

    for (auto it = args.begin(), end = args.end(); it != end; ++it)
    {
        if (*it == "-h" || *it == "--help")
        {
            options.show_help = true;
        }
        else if (*it == "-o" || *it == "--output")
        {
            if (++it != end)
            {
                options.output_filename = std::string(*it);
            }
            else
            {
                return std::unexpected(
                    ParseArgsError::missing_argument);
            }
        }
        else if (/* otra opción */)
        {
            // ...
        }
        else if (!it->starts_with("-"))
        {
            options.additional_args.push_back(std::string(*it));
        }
        else
        {
            return std::unexpected(
                ParseArgsError::unknown_option);
        }
    }
}

```

④

⑤

⑥

```
    return options; ⑦
}
```

- ① Enumeración con los posibles errores al procesar los argumentos de la línea de comandos.
- ② Estructura con las opciones admitidas por el programa.
- ③ Si el programa admite argumentos adicionales que no son opciones, se pueden guardar en un vector.
- ④ La función `parse_arguments()` devuelve un `std::expected` que puede contener una estructura `ProgramOptions` en caso de éxito o un `ParseArgsError` con el motivo del error en caso de fallo.
- ⑤ En caso de que se indique la opción `-o` o `--output` pero no se indique el nombre del archivo de salida, se devuelve `ParseArgsError::missing_argument` para notificar el error a la función que ha invocado a `parse_arguments()`. Este valor se marca como `std::unexpected` para indicar que se ha producido un error.
- ⑥ Si se encuentra una opción desconocida, se devuelve el código de error `unknown_option`. Este valor se marca como `std::unexpected` para indicar que se ha producido un error.
- ⑦ En caso de éxito, se devuelve la estructura `ProgramOptions` con las opciones encontradas y sus argumentos.

La función `parse_arguments()` devuelve un `std::expected` para poder devolver la estructura de datos `ProgramOptions` en caso de éxito o un `enum ParseArgsError` con el código de error en caso de fallo.

Esta función se puede invocar fácilmente desde `main()`:

```
int main(int argc, char* argv[])
{
    auto options = parse_arguments(argc, argv); ①

    if (! options.has_value()) ②
    {
        if (options.error() == ParseArgsError::missing_argument) ③
        {
            std::println(stderr, "error: falta argumento para opción");
        }
        else if (options.error() == ParseArgsError::unknown_option)
        {
            std::println(stderr, "error: opción desconocida");
        }

        print_usage();
        return EXIT_FAILURE; ④
    }

    if (options.value().show_help) ⑤
    {
```

```

        print_usage();
        return EXIT_SUCCESS;
    }

    if (!options.value().output_filename.empty())           ⑥
    {
        std::println("Archivo de salida: {}",
                     options.value().output_filename);
    }

    // Procesar argumentos adicionales
    for (const auto& arg : options.value().additional_args) ⑦
    {
        std::println("Procesando: {}", arg);
    }

    return EXIT_SUCCESS;
}

```

- ① Llamar a `parse_arguments()` para procesar los argumentos de línea de comandos en `argc` y `argv`. El objeto devuelto en `options` es de tipo `std::expected<ProgramOptions, ParseArgsError>`.
- ② Comprobar si `parse_arguments()` terminó con éxito, comprobando el objeto `std::expected` devuelto. Si `options.has_value()` devuelve `false`, es que ha habido un error y lo que guarda `options` no es un `ProgramOptions` sino un `ParseArgsError`.
- ③ Comprobar el motivo del error para mostrar un mensaje adecuado.
- ④ En caso de error, terminar el programa con un código de salida diferente de 0.
- ⑤ En caso de éxito al procesar los argumentos de la línea de comandos, se accede a la estructura `ProgramOptions` con `options.value()`. Como se ilustra en el ejemplo, así se puede acceder a `ProgramOptions::show_help` para comprobar si el usuario indicó que quería leer la ayuda del programa.
- ⑥ Comprobar si se indicó un nombre de archivo de salida y mostrarlo por pantalla.
- ⑦ Procesar los argumentos adicionales indicados en la línea de comandos (no opciones).

A.3. Procesamiento con `getopt()`

Como alternativa al procesamiento manual de argumentos, POSIX proporciona la función `getopt()` que simplifica el análisis de opciones de línea de comandos siguiendo las convenciones estándar de UNIX, y permitiendo obtener una interfaz más consistente con la de otros programas.

```
int getopt(int argc, char *const argv[], const char *optstring);
```

A.3.1. Variables globales de `getopt()`

La función `getopt()` utiliza y modifica varias variables globales definidas en el archivo `<unistd.h>`:

- optind** Índice del siguiente elemento a procesar en `argv`. Inicialmente tiene el valor 1 (el primer argumento después del nombre del programa). Después de procesar todas las opciones, `optind` apunta al primer argumento que no es una opción.
- optopt** Carácter de la opción que causó un error (cuando `getopt()` retorna '?').
- optarg** Puntero al argumento de una opción que requiere argumento. Por ejemplo, si la opción es `-o archivo`, `optarg` apuntará a `"archivo"`.
- opterr** Si es diferente de 0 (valor por defecto), `getopt()` imprime mensajes de error automáticamente a `stderr`. Se puede establecer a 0 para desactivar estos mensajes y manejar los errores manualmente.

A.3.2. Cadena de opciones `optstring`

El parámetro `optstring` especifica las opciones válidas y si requieren argumentos:

- **Letra sola:** La opción no requiere argumento.
 - Ejemplo: `"abc"` acepta `-a`, `-b`, `-c`.
- **Letra seguida de :** La opción requiere un argumento.
 - Ejemplo: `"a:bc"` acepta `-a archivo`, `-b`, `-c`.
- **Letra seguida de ::** La opción tiene un argumento opcional.
 - Ejemplo: `"a::bc"` acepta `-a`, `-a archivo`, `-b`, `-c`.

Por ejemplo, para un programa con opciones `-h` (sin argumento) y `-o` (con argumento), la cadena sería: `"ho:"`.

i Nota

Para soportar opciones largas (`--help`, `--output`, etc.) en programas se debe usar `getopt_long()`, una extensión GNU de `getopt()`.

A.3.3. Valor de retorno

La función `getopt()` retorna:

- El **carácter de la opción** encontrada (por ejemplo, `'h'`, `'o'`).
- `'?'` si encuentra una opción desconocida o falta un argumento requerido.
- `-1` cuando no quedan más opciones por procesar.

A.3.4. Uso básico de getopt()

El patrón típico es llamar a `getopt()` en un bucle hasta que retorne `-1`:

```
for (int opt; (opt = getopt(argc, argv, "ho:")) != -1;)
{
    switch (opt)
    {
        case 'h':
            show_help = true;
            break;

        case 'o':
            output_filename = optarg;
            break;

        case '?':
            return EXIT_FAILURE;
    }
}

if (optind < argc)
{
    for (int i = optind; i < argc; i++)
    {
        // Procesar argumentos adicionales:
        // (argv[optind], argv[optind+1], ...)
    }
}
```

- ① En cada iteración, `opt` contiene el carácter de la opción encontrada. Se utiliza un `switch` para procesar cada opción según su carácter.
- ② Caso para la opción `-h` que activa la ayuda.
- ③ Caso para la opción `-o` que usa `optarg` para obtener el argumento de la opción.
- ④ Caso para manejar errores cuando se encuentra una opción desconocida o falta un argumento requerido. Aquí se puede mostrar un mensaje de error personalizado si la variable global `opterr` vale 0.
- ⑤ Después de procesar todas las opciones, `optind` apunta al primer argumento que no es una opción. Si `optind < argc`, significa que hay argumentos adicionales que se pueden procesar.

A.3.5. Implementación de completa de `parse_arguments()` con `getopt()`

Ahora podemos adaptar el ejemplo anterior de `parse_arguments()` para usar `getopt()`:

```
std::expected<ProgramOptions, ParseArgsError>
parse_arguments(int argc, char* argv[])
{
    ProgramOptions options;
    opterr = 0; ①

    for (int opt; opt = getopt(argc, argv, "ho:") != -1;)
    {
        switch (opt)
        {
            case 'h':
                options.show_help = true;
                break;

            case 'o':
                options.output_filename = optarg;
                break;

            case '?':
                if (optopt == 'o') ②
                {
                    // Falta el argumento de -o
                    return std::unexpected(
                        ParseArgsError::missing_argument);
                }
                else
                {
                    // Opción desconocida
                    return std::unexpected(
                        ParseArgsError::unknown_option);
                }

            default:
                return std::unexpected(
                    ParseArgsError::unknown_option);
        }
    }

    if (optind < argc) ③
    {
        for (int i = optind; i < argc; i++)
        {
            options.additional_args.push_back(argv[i]);
        }
    }
}
```



```
    return options;
}
```

- ① Desactivar los mensajes de error automáticos de `getopt()` para procesar los errores manualmente.
- ② Comprobar si el error se debe a que falta el argumento de la opción `-o` o si es una opción desconocida. Según el caso, se devuelve el código de error adecuado.
- ③ Después de procesar todas las opciones, usar `optind` para recoger los argumentos adicionales no-opción y guardarlos en `options.additional_args`.

El uso de esta versión de `parse_arguments()` en `main()` es idéntico al ejemplo anterior:

```
int main(int argc, char* argv[])
{
    auto options = parse_arguments(argc, argv);

    if (!options.has_value())
    {
        if (options.error() == ParseArgsError::missing_argument)
        {
            std::println(stderr, "error: falta argumento para opción");
        }
        else if (options.error() == ParseArgsError::unknown_option)
        {
            std::println(stderr, "error: opción desconocida");
        }

        print_usage();
        return EXIT_FAILURE;
    }

    if (options.value().show_help)
    {
        print_usage();
        return EXIT_SUCCESS;
    }

    // ... resto del código ...
}
```

B. Gestión de procesos

B.1. Creación de procesos con `fork()`

La función `fork()` se utiliza para crear un nuevo proceso (hijo) duplicando el proceso que la llama (proceso padre):

```
pid_t fork();
```

El nuevo proceso tiene un espacio de direcciones independiente y comienza a ejecutarse desde el punto donde se llamó a `fork()`, pero tiene respecto al padre:

- El mismo código
- La misma memoria (copiada, no compartida)
- Los mismos descriptores de archivo abiertos
- El mismo directorio de trabajo
- Las mismas variables de entorno

Según el valor retornado por `fork()`, se puede determinar si el código se está ejecutando en el proceso padre o en el proceso hijo:

- En el **proceso padre**, retorna el PID del hijo (valor positivo).
- En el **proceso hijo**, retorna 0.
- En caso de **error**, no se crea el hijo, retorna -1 y establece `errno` en el padre.

A continuación se muestra un ejemplo básico de uso de `fork()`:

```
pid_t pid = fork();

if (pid == -1)
{
    // Error al crear el proceso...
}
else if (pid == 0)
{
    // En el proceso hijo...

    std::println("Soy el hijo, mi PID es {}", getpid());
    std::println("Mi padre tiene PID {}", getppid());

    std::exit(EXIT_SUCCESS);
}
else
{
    // En el proceso padre...
```

①

```

std::println("Soy el padre, mi PID es {}", getpid());
std::println("Creé un hijo con PID {}", pid);

wait(nullptr);
}

```

- ① El proceso hijo termina con `std::exit()` para evitar ejecutar que salga del `if` y ejecute el resto del código.
- ② El proceso padre espera a que el hijo termine con `wait()`. Esto es importante para evitar que el proceso hijo quede como zombie cuando termine.

B.1.1. Descriptores de archivo compartidos

Como hemos comentado, los descriptores de archivo abiertos antes de `fork()` son **compartidos** entre padre e hijo en el sentido de que ambos tienen copias de los descriptores que apuntan al **mismo recurso subyacente** (archivo, tubería, *socket*, etc.).

Esto significa que:

- Ambos pueden leer/escribir en el mismo archivo.
- La posición de lectura/escritura es compartida, por lo que una operación de lectura/escritura en uno mueve la posición para el otro.
- Ambos deben cerrar sus descriptores independientemente.

Por ejemplo, si ejecutamos el siguiente código el archivo contendrá ambos mensajes escritos por padre e hijo, según el orden de ejecución. Padre e hijo no se sobrescriben mutuamente porque usan el mismo descriptor compartido, por lo que cada escritura avanza la posición en el archivo para los dos:

```

int fd = open("archivo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
pid_t pid = fork();

if (pid == 0)
{
    // El proceso hijo escribe en el archivo...

    write(fd, "Hijo\n", 5);
    close(fd);

    std::exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // El proceso padre escribe en el archivo...
}

```

```
    write(fd, "Padre\n", 6);  
    close(fd);  
  
    wait(nullptr);  
}
```

- ① Abrir un archivo antes de `fork()`, obteniendo un descriptor `fd`.
- ② El proceso hijo escribe en el archivo usando `fd`.
- ③ El proceso padre escribe en el archivo usando `fd`.
- ④ El padre espera a que el hijo termine.

B.2. Reemplazo de programa con `exec()`

La familia de funciones `exec()` se utiliza para **reemplazar** el código del proceso actual con un nuevo programa.

```
int execl(const char *path, const char *arg, ...);  
int execlp(const char *file, const char *arg, ...);  
int execl_e(const char *path, const char *arg, ..., char *const envp[]);  
int execv(const char *path, char *const argv[]);  
int execvp(const char *file, char *const argv[]);  
int execve(const char *path, char *const argv[], char *const envp[]);
```

Estas funciones:

- **No retornan si tiene éxito:** El programa actual es completamente reemplazado por el nuevo programa, por lo que no vuelven al código que las llamó.
- **Solo retornan si hay error:** Retornan `-1` y establecen `errno`.
- **Los descriptores abiertos se mantienen:** Los descriptores de archivo abiertos antes de `exec()` permanecen abiertos en el nuevo programa (a menos que se obtuvieran con el flag `O_CLOEXEC`).
- **Variables de entorno heredadas:** El nuevo programa hereda las variables de entorno del proceso actual (excepto en las variantes de `exec` con `e`).

B.2.1. Diferencias entre las variantes de `exec()`

Kas distintas variantes de la función `exec()` permiten pasar argumentos y variables de entorno de diferentes maneras:

B.2.1.1. forma de pasar argumentos

exec1(), execlp(), execl() Permiten pasar los argumentos al programa como una **lista variable** de argumentos separados por coma. El último argumento debe ser **nullptr**.

```
exec1("/bin/ls", "ls", "-l", "/etc", nullptr);
```

execv(), execlp(), execl() Permiten pasar los argumentos al programa como un **array de cadenas** de caracteres. Esta versión es útil cuando el número de argumentos no se conoce en tiempo de compilación. El array debe terminar con **nullptr**.

```
char* args[] = { "ls", "-l", "/etc", nullptr };  
execv("/bin/ls", args);
```

B.2.1.2. Por búsqueda en PATH

exec1(), execv(), execl(), execlp(), execl() Requieren la **ruta completa** (absoluta o relativa) del programa a ejecutar.

```
exec1("/bin/ls", "ls", "-l", nullptr);
```

```
execv("/usr/bin/gcc", args);
```

execlp(), execlp() Si no se especifica la ruta completa, buscan el programa en los directorios especificados en la variable de entorno PATH.

```
execlp("ls", "ls", "-l", nullptr); ①
```

① Busca el ejecutable ls en PATH y lo ejecuta si lo encuentra.

```
execvp("gcc", args); ①
```

① Busca el ejecutable gcc en PATH y lo ejecuta si lo encuentra.

B.2.1.3. Por variables de entorno

exec1(), execlp(), execv(), execlp() El nuevo programa **hereda** las variables de entorno del proceso actual.

execl(), execlp() Permiten especificar un array de **variables de entorno personalizadas** para el nuevo programa.

```
char* env[] = {  
    "LANG=en_US.UTF-8",  
    "PATH=/usr/bin:/bin",  
    nullptr  
};  
execl("/bin/ls", "ls", "-l", nullptr, env); ①
```

- ① Array con las variables de entorno con las que se ejecutará `/bin/ls`.

```
char* args[] = { "ls", "-l", nullptr };
char* env[] = {
    "LANG=en_US.UTF-8",
    "PATH=/usr/bin:/bin",
    nullptr
};
execve("/bin/ls", args, env);
```

①

- ① Array con las variables de entorno con las que se ejecutará `/bin/ls`.

B.2.2. Ejemplo con `execvp()`

La variante más común y conveniente es `execvp()` porque busca en `PATH` y acepta arrays, que se pueden construir dinámicamente durante la ejecución.

```
pid_t pid = fork();

if (pid == 0)
{
    // El proceso hijo ejecuta el comando ls -l /tmp

    char* args[] = {
        const_cast<char*>("ls"),
        const_cast<char*>("-l"),
        const_cast<char*>("/tmp"),
        nullptr
    };

    execvp("ls", args);

    std::println(stderr, "Error al ejecutar ls: {}",
        std::strerror(errno));
    std::exit(EXIT_FAILURE);
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
```

①

②

③

④

⑤

⑥

⑦

```

        std::println("El comando terminó con código {}",
                     WEXITSTATUS(status));
    }
}

```

- ① `argv[0]` debe ser el nombre del comando porque por convención, en los sistemas POSIX el primer argumento es siempre el nombre del programa.
- ② Los literales de cadena como `"ls"` son de tipo `const char*` porque no son modificables, pero los argumentos de `exec()` están declarados como `char*` por razones históricas. Tenemos que usar el operador `const_cast<char*>()` para quitar el `const`.
- ③ Es importante recordar que el array siempre debe terminar en `nullptr`, para que `exec()` sepa dónde termina la lista de argumentos.
- ④ Llamada a `execvp()` para ejecutar el comando `ls -l /tmp`.
- ⑤ Siempre verificar si `exec()` retorna, ya que indica un error. Para ser exactos, `exec()` retorna -1 en caso de error y establece `errno` con el código de error correspondiente.
- ⑥ Mientras tanto, el proceso padre espera a que el hijo, donde se ejecuta `ls -l /tmp`, termine. Al terminar, `waitpid()` llena la variable `status` con información sobre la terminación del hijo.
- ⑦ Se verifica si el hijo terminó normalmente.
- ⑧ Se obtiene el código de salida del hijo con `WEXITSTATUS()`, que es el valor que retornó `ls` al terminar. Por convención, un código de salida 0 indica que el comando se ejecutó correctamente, mientras que un valor distinto de 0 indica algún tipo de error.

B.3. Espera de procesos hijos con `wait()`

Las funciones `wait()` y `waitpid()` se utilizan para hacer que el proceso padre espere a que termine uno de sus procesos hijos.

```

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);

```

En concreto:

`wait(status)` Bloquea el proceso hasta que **cualquier** proceso hijo termine. Retorna el PID del hijo que terminó, o -1 si ocurrió un error.

`waitpid(pid, status, options)` Espera a un hijo **específico** con el PID dado termine.

- El argumento `options` permite modificar el comportamiento de la espera. Si `options` es 0, se comporta igual que `wait()`, bloqueando el proceso y retornando el PID del hijo que terminó.
- Con `options = WNOHANG`, el proceso no se bloquea si el hijo no ha terminado, retornando 0 en ese caso. Esto para comprobar periódicamente si un hijo ha terminado sin bloquear el proceso padre.

B.3.1. Interpretar el estado de salida

Al volver de `wait()` o `waitpid()`, el parámetro `status` contiene información sobre la terminación del proceso hijo. Se usan las siguientes macros para interpretarlo:

WIFEXITED(status) Retorna `true` si el hijo terminó normalmente (con `std::exit()` o retorno de `main()`).

WEXITSTATUS(status) Si **WIFEXITED** es `true`, retorna el código de salida (0-255) del hijo.

WIFSIGNALED(status) Retorna `true` si el hijo fue terminado por una señal.

WTERMSIG(status) Si **WIFSIGNALED** es `true`, retorna el número de señal que terminó el hijo.

Por ejemplo, para capturar el código de salida de un hijo que termina normalmente:

```
pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo...
    sleep(300);

    std::exit(42);
}
else
{
    // En el proceso padre...

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        int exit_code = WEXITSTATUS(status);
        std::println("Hijo terminó con código {}", exit_code);
    }
    else if (WIFSIGNALED(status))
    {
        int signal = WTERMSIG(status);
        std::println("Hijo terminado por señal {}", signal);
    }
}
```

- ① El proceso hijo termina con código 42.
- ② El proceso padre espera a que el hijo termine y obtiene su estado.
- ③ Se verifica si el hijo terminó normalmente y se obtiene su código de salida (42) para mostrarlo por pantalla.

- ④ Se verifica si el hijo fue terminado por una señal y se obtiene el número de la señal. Por ejemplo, si el hijo fuera terminado con `kill -15 <pid>` mientras espera en `sleep(300)`, se mostraría el mensaje: “Hijo terminado por señal 15”.

B.3.2. Ejemplo con comando externo

Como vimos antes, se puede ejecutar un comando externo y obtener su código de salida, que puede indicar si el comando se ejecutó correctamente o si hubo algún error y cuál fue:

```
pid_t pid = fork();

if (pid == 0)
{
    // El proceso hijo ejecuta el comando ls -l /etc

    execl("/bin/ls", "ls", "-l", "/etc", nullptr);

    std::exit(128);
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        int exit_code = WEXITSTATUS(status);

        if (exit_code == 0)
        {
            std::println("El comando 'ls' se ejecutó correctamente");
        }
        else if (exit_code == 128)
        {
            std::println(stderr, "Error al ejecutar 'ls'");
        }
        else
        {
            std::println(stderr, "'ls' terminó con código {}", exit_code);
        }
    }
}
```

- ① Si `exec1()` falla, el hijo termina con código 128 para indicar el error. Esto es exactamente lo mismo que hace BASH, que en caso de error al ejecutar un comando retorna un valor alto (>125) para diferenciar errores al ejecutar el comando de los códigos de salida del propio comando.
- ② El padre verifica si el hijo terminó con código 0 para informar que `ls` se ejecutó correctamente.
- ③ El padre verifica si el hijo terminó con código 128 para informar del error al intentar ejecutar `ls` y distinguirlo de los códigos de salida normales de `ls`.

B.3.3. Importancia de `wait()`

Es **fundamental** llamar a `wait()` o `waitpid()` por cada hijo creado para:

1. **Evitar procesos zombie:** Si el padre no hace `wait()`, los hijos terminados quedan como zombies, ocupando entradas en la tabla de procesos del sistema.
2. **Liberar recursos:** El sistema libera completamente los recursos del hijo después de `wait()`.
3. **Verificar el éxito:** El código de salida indica si el comando ejecutado tuvo éxito.

B.4. Redirección de entrada/salida

La función `dup2()` duplica un descriptor de archivo en un número de descriptor específico, lo que es útil para redirigir la entrada/salida estándar de un proceso a un archivo o tubería:

```
int dup2(int oldfd, int newfd);
```

Donde:

oldfd Descriptor del archivo o recurso origen al que se quiere redirigir.

newfd Número de descriptor destino, que es el descriptor redirigido.

La función `dup2()` cierra `newfd` si ya está abierto luego copia `oldfd` a `newfd`, de modo que ambos descriptors apunten al mismo recurso. Después de la llamada, cualquier operación de lectura/escritura en `newfd` afectará al mismo recurso que `oldfd`. Si `oldfd` no es válido, o si ocurre algún otro error, `dup2()` retorna `-1` y establece `errno` con el código del error.

B.4.1. Ejemplos de redirección a archivos

El uso principal de `dup2()` es redirigir la entrada estándar (`stdin`), salida estándar (`stdout`), o salida de error (`stderr`) de un proceso a archivos:

B.4.1.1. Redirigir la salida estándar a un archivo

```
int fd = open("salida.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644); ①
if (fd == -1)
{
    // Error al abrir archivo...
}

dup2(fd, STDOUT_FILENO); ②
close(fd);

std::println("Este mensaje se guarda en salida.txt"); ③
write(STDOUT_FILENO, "También este\n", 13);
```

- ① Abrir (o crear) un archivo para escribir la salida.
- ② Redirigir `stdout` al archivo usando `dup2()`. Una vez redirigido, ya no es necesario el descriptor original `fd`, por lo que se cierra.
- ③ Cualquier salida a `stdout` (como `std::println` o `write`) se escribirá en `salida.txt` en lugar de en la consola.

B.4.1.2. Redirigir la entrada estándar desde un archivo

```
int fd = open("entrada.txt", O_RDONLY); ①
if (fd == -1)
{
    // Error al abrir archivo...
}

dup2(fd, STDIN_FILENO); ②
close(fd);

std::string line;
std::getline(std::cin, line); ③
std::println("Leído de 'entrada.txt': {}", line);
```

- ① Abrir un archivo para leer la entrada.
- ② Redirigir `stdin` al archivo usando `dup2()`. Una vez redirigido, ya no es necesario el descriptor original `fd`, por lo que se cierra.
- ③ Cualquier lectura de `stdin` (como `std::cin` o `read`) leerá desde `entrada.txt` en lugar del teclado.

B.4.2. Redirección con `fork()` y `exec()`

La redirección es especialmente útil combinada con `fork()` y `exec()` para ejecutar comandos con entrada/salida redirigida:

B.4.2.1. Ejecutar comando con salida redirigida

Por ejemplo, para ejecutar `ls` de forma equivalente al comando de *shell* `ls -l > listado.txt`:

```
pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo, se hará la redirección y se ejecuta el comando

    int fd = open("listado.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644); ①

    dup2(fd, STDOUT_FILENO); ②
    close(fd);

    execlp("ls", "ls", "-l", nullptr); ③

    std::exit(128); ④
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0); ⑤

    //...
}
```

- ① En el procesos hijo –que es dónde se hará la redirección– se abre el archivo `listado.txt` para escribir.
- ② Luego se redirige `stdout` al archivo usando `dup2()`, y se cierra el descriptor original.
- ③ Finalmente, se ejecuta `ls -l` con una función de la familia `exec()`. La salida de `ls` se escribirá en `listado.txt` en lugar de la consola.
- ④ Si `execlp()` falla, el hijo termina con código 128 para indicar el error al ejecutar el comando.
- ⑤ El proceso padre espera a que el hijo termine y obtiene su estado para verificar si hubo errores.

B.4.2.2. Ejecutar comando con entrada redirigida

Por ejemplo, para ejecutar `wc` de forma equivalente al comando de *shell* `wc -l < datos.txt`:

```
pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo, se hará la redirección y se ejecuta el comando

    int fd = open("datos.txt", O_RDONLY);           ①

    dup2(fd, STDIN_FILENO);                         ②
    close(fd);

    execlp("wc", "wc", "-l", nullptr);             ③

    std::exit(128);                                 ④
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0);                      ⑤

    //...
}
```

- ① En el procesos hijo –que es dónde se hará la redirección– se abre el archivo `datos.txt` para leer.
- ② Luego se redirige `stdin` al archivo usando `dup2()`, y se cierra el descriptor original.
- ③ Finalmente, se ejecuta `wc -l` con una función de la familia `exec()`. La entrada de `wc` se leerá desde `datos.txt` en lugar del teclado.
- ④ Si `execlp()` falla, el hijo termina con código 128 para indicar el error al ejecutar el comando.
- ⑤ El proceso padre espera a que el hijo termine y obtiene su estado para verificar si hubo errores.

C. Tuberías anónimas

Como se explicó en la la Unidad 2, las tuberías son un mecanismo de comunicación entre procesos que utiliza la misma interfaz que empleamos para manipular archivos: se envían datos mediante `write()` en un extremo y se reciben mediante `read()` en el otro.

Existen dos tipos de tuberías:

Tuberías sin nombre o anónimas Se crean mediante `pipe()`. Se utilizan para la comunicación entre procesos relacionados, como un proceso padre y sus hijos.

Tuberías con nombre (FIFO) Se crean en el sistema de archivos mediante `mkfifo()`. Se utilizan para la comunicación entre procesos no relacionados que se ejecutan de forma independiente. Estas ya fueron explicadas en la Unidad 2.

En esta sección nos centraremos en las **tuberías anónimas** y cómo utilizarlas junto con `fork()` y `exec()` para comunicar procesos.

C.1. Creación de tuberías anónimas

Para crear una tubería anónima, se utiliza la función `pipe()`:

```
int pipe(int pipefd[2]);
```

Donde:

pipefd Array de dos elementos que se rellenará con los descriptors de la tubería:

- `pipefd[0]`: Descriptor para **leer** en la tubería
- `pipefd[1]`: Descriptor para **escribir** en la tubería

La función retorna:

- 0 si tiene éxito.
- -1 si hay error, estableciendo `errno`.

Por ejemplo, el siguiente código crea una tubería anónima e imprime los descriptors obtenidos:

```
int pipefd[2];

int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
}

std::println("Descriptor para lectura: {}", pipefd[0]);
std::println("Descriptor para escritura: {}", pipefd[1]);
```

C.1.1. Uso básico de tuberías anónimas

Una vez creada la tubería, se puede escribir datos en un extremo y leerlos desde el otro, como hicimos con los FIFOs en la Unidad 2. Por ejemplo:

```
int pipefd[2];  
int result = pipe(pipefd);  
if (result == -1)  
{  
    // Error al crear la tubería...  
}  
  
// Escribir datos en la tubería  
std::string_view mensaje = "Hola desde la tubería";  
ssize_t bytes_written = write(pipefd[1], mensaje.data(), mensaje.size());  
if (bytes_written == -1)  
{  
    // Error al escribir en la tubería...  
}  
  
// Leer datos de la tubería  
std::array<char, 100> buffer;  
ssize_t bytes_read = read(pipefd[0], buffer.data(), buffer.size());  
if (bytes_read == -1)  
{  
    // Error al leer de la tubería...  
}  
  
buffer[bytes_read] = '\0';  
std::println("Mensaje leído: {}", buffer.data());  
  
// Cerrar los descriptores  
close(pipefd[0]);  
close(pipefd[1]);
```

- ① Crear la tubería, obteniendo los descriptores de lectura y escritura.
- ② Escribir un mensaje en el extremo de escritura de la tubería.
- ③ Leer el mensaje desde el extremo de lectura de la tubería.
- ④ Añadir un terminador nulo para tratar el buffer como una cadena. El terminador nulo no se envió porque solo se enviaron los bytes de la cadena de texto.
- ⑤ Cerrar ambos descriptores de la tubería cuando ya no se necesitan.

i Nota

Como se explicó en la Unidad 2, la lectura y escritura en tuberías funciona igual que en archivos, utilizando `read()` y `write()`, tal como las usamos en la Unidad 1. Primero es fundamental recordar que estas funciones pueden leer o escribir menos bytes de los solicitados, por lo que es recomendable utilizar bucles para garantizar que se procesan todos los datos. En este ejemplo simplificado, asumimos que el mensaje cabe en una sola operación de lectura/escritura, pero en aplicaciones reales, se deben

manejar estos casos adecuadamente.

También debemos tener en cuenta que `read()` en una tubería se bloquea si no hay datos disponibles, esperando hasta que se escriban datos en el extremo de escritura o se cierren todos los descriptores de escritura –en cuyo caso devuelve 0, indicando el final de archivo (EOF)–. De forma similar, `write()` puede bloquearse si el buffer interno de la tubería está lleno, esperando hasta que haya espacio disponible.

Este comportamiento de bloqueo es una característica fundamental para la sincronización entre procesos.

C.2. Comunicación entre procesos con tuberías

Los descriptores de archivo abiertos se **heredan de procesos padre a procesos hijo** cuando se utiliza `fork()` (ver la Sección B.1).

Esto significa que si se crea una tubería antes de llamar a `fork()`, tanto el proceso padre como el proceso hijo tendrán acceso a los descriptores de la tubería y podrán comunicarse entre ellos, permitiendo a uno de los procesos escribir datos en la tubería y al otro leerlos. Para ilustrarlo, vamos a trasladar el ejemplo anterior a un escenario con `fork()`:

```
int pipefd[2];  
int result = pipe(pipefd);  
if (result == -1)  
{  
    // Error al crear la tubería...  
}  
  
pid_t pid = fork();  
  
if (pid == 0)  
{  
    // En el proceso hijo...  
  
    close(pipefd[1]);  
  
    std::array<char, 100> buffer;  
    ssize_t bytes_read = read(pipefd[0], buffer.data(), buffer.size());  
    if (bytes_read == -1)  
    {  
        // Error al leer de la tubería...  
    }  
  
    buffer[bytes_read] = '\0';  
    std::println("El hijo recibió: {}", buffer.data());  
}
```



```

    close(pipefd[0]);
    std::exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // En el proceso padre...

    close(pipefd[0]); ③

    // Enviar mensaje al hijo
    std::string_view mensaje = "Hola desde el padre";
    ssize_t bytes_written = write(pipefd[1], mensaje.data(), mensaje.size()); ⑤
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
    }

    close(pipefd[1]);
    wait(nullptr); ⑥
}
else
{
    // Error al crear el proceso hijo...
}

```

- ① Crear la tubería antes de llamar a `fork()`, para que ambos procesos la hereden.
- ② Crear un proceso hijo con `fork()`.
- ③ Cada proceso cierra el extremo de la tubería que no va a usar (el hijo cierra el de escritura y el padre cierra el de lectura).
- ④ El hijo lee datos de la tubería.
- ⑤ El padre escribe datos en la tubería.
- ⑥ El padre espera a que el hijo termine con `wait()`. El puntero `nullptr` indica que no nos interesa el estado de salida del hijo en este caso.

! Importancia de cerrar extremos no usados

Es **fundamental** que cada proceso cierre los extremos de la tubería que no va a usar:

- El proceso que solo escribe debe cerrar `pipefd[0]` (lectura).
- El proceso que solo lee debe cerrar `pipefd[1]` (escritura).

Esto es necesario para que:

1. **Se detecte el fin de archivo (EOF):** Cuando todos los descriptores de escritura se cierran, `read()` en el extremo de lectura retorna 0, indicando EOF.

2. **Se eviten fugas de descriptors:** Los descriptors abiertos consumen recursos del sistema.
3. **Funcione la detección de tubería rota:** Si el lector cierra su extremo y el escritor intenta escribir, recibirá la señal SIGPIPE.

C.3. Redirección con tuberías

Como los descriptors de tuberías son descriptors de archivo normales, se pueden usar con `dup2()` (ver la Sección B.4) para redirigir la entrada o salida estándar de un proceso hacia o desde una tubería. Esto es especialmente útil cuando se combina con `fork()` y `exec()` (ver las Secciones B.1 y B.2) para ejecutar programas externos que leen de `stdin` o escriben en `stdout`.

C.3.1. Redirigir stdout a una tubería

En este patrón, el hijo redirige su salida estándar a una tubería para que el padre pueda leer lo que el hijo imprime:

```
int pipefd[2];
int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
}

pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo, su salida estándar irá a la tubería

    close(pipefd[0]);                                ①

    dup2(pipefd[1], STDOUT_FILENO);                  ②
    close(pipefd[1]);                                ③

    // Todo lo que el hijo escriba a stdout irá a la tubería
    std::println("Mensaje del hijo");

    std::exit(EXIT_SUCCESS);
}
else if (pid > 0)
{

```

```

    // En el proceso padre, leer de la tubería

    close(pipefd[1]); ①

    std::array<char, 100> buffer;
    ssize_t bytes_read = read(pipefd[0], buffer.data(), buffer.size()); ④
    if (bytes_read == -1)
    {
        // Error al leer de la tubería...
    }

    buffer[bytes_read] = '\0';
    std::println("El padre recibió: {}", buffer.data());

    close(pipefd[0]);
    wait(nullptr);
}
else
{
    // Error al crear el proceso hijo...
}

```

- ① Cada proceso cierra el extremo de la tubería que no va a usar (el hijo cierra el de lectura y el padre cierra el de escritura).
- ② El hijo redirige su salida estándar al extremo de escritura de la tubería.
- ③ El hijo cierra el descriptor original de la tubería, porque una vez redirigido ya no es necesario.
- ④ El padre lee los datos escritos por el hijo desde la tubería, que son los que el hijo envió a su salida estándar.

C.3.2. Redirigir stdin desde una tubería

En este patrón, el hijo redirige su entrada estándar a una tubería para leer lo que el padre escribe:

```

int pipefd[2];
int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
}

pid_t pid = fork();

```

```

if (pid == 0)
{
    // En el proceso hijo, su entrada estándar vendrá de la tubería

    close(pipefd[1]);                                ①

    dup2(pipefd[0], STDIN_FILENO);                  ②
    close(pipefd[0]);                                ③

    std::string buffer;
    std::getline(std::cin, buffer);                  ④

    std::println("Hijo recibió de stdin: {}", buffer);

    std::exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // En el proceso padre, escribir en la tubería

    close(pipefd[0]);                                ①

    std::string_view mensaje = "Datos para el hijo";
    ssize_t bytes_written = write(pipefd[1], mensaje.data(), mensaje.size()); ⑤
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
    }

    close(pipefd[1]);                                ⑥
    wait(nullptr);
}
else
{
    // Error al crear el proceso hijo...
}

```

- ① Cada proceso cierra el extremo de la tubería que no va a usar (el hijo cierra el de escritura y el padre cierra el de lectura).
- ② El hijo redirige su entrada estándar al extremo de lectura de la tubería.
- ③ El hijo cierra el descriptor original de la tubería, porque una vez redirigido ya no es necesario.
- ④ El hijo lee una línea desde su entrada estándar (la tubería). En este caso utilizar `std::getline()` para leer una línea completa, pero también se podría usar

```
read(STDIN_FILENO, ...).
```

- ⑤ El padre escribe datos en el extremo de escritura de la tubería.
- ⑥ El padre cierra el extremo de escritura. Esto envía EOF (`read()` retorna 0) al hijo al no haber más descriptores de escritura abiertos.

C.4. Patrón completo: Pipe + Fork + Exec + Redirección

El patrón más potente en programación Unix combina tuberías, `fork()`, `exec()` y redirecciones para ejecutar un programa externo y comunicarse con él. Este patrón es la base del operador `|` en la *shell*, que conecta la salida de un comando con la entrada de otro.

```
int pipefd[2];
int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
}

pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo, su entrada estándar vendrá de la tubería

    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    // Ejecutar en el proceso hijo el comando que leerá de stdin
    execlp("wc", "wc", "-w", nullptr);

    // Si llegamos aquí, exec() falló
    std::println(stderr, "Error: {}", std::strerror(errno));
    std::exit(EXIT_FAILURE);
}
else if (pid > 0)
{
    // En el proceso padre, escribimos en la tubería

    close(pipefd[0]);

    std::string_view texto = "Hola mundo desde una tubería\n";
    ssize_t bytes_written = write(pipefd[1], texto.data(), texto.size());
    if (bytes_written == -1)
```

```

{
    // Error al escribir en la tubería...
}

close(pipefd[1]);

int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
{
    std::println("Comando ejecutado con éxito");
}
else
{
    std::println(stderr, "Comando falló");
}
}
else
{
    // Error al crear el proceso hijo...
}

```

- ① Crear la tubería antes de llamar a `fork()`, para que ambos procesos tengan acceso a ella.
- ② Crear un proceso hijo con `fork()`.
- ③ Cada proceso cierra el extremo de la tubería que no va a usar (el hijo cierra el de escritura y el padre cierra el de lectura).
- ④ El hijo redirige su `stdin` al extremo de lectura de la tubería.
- ⑤ El hijo cierra el descriptor original (ya no es necesario).
- ⑥ El hijo ejecuta `wc -w` que leerá de su entrada estándar que, por la redirección, es la salida de la tubería.
- ⑦ Si `exec()` falla, se imprime un mensaje de error.
- ⑧ El padre escribe texto en el extremo de escritura de la tubería.
- ⑨ El padre cierra el extremo de escritura. Esto envía EOF al hijo, al no haber más descriptores de escritura abiertos.
- ⑩ El padre espera a que el hijo termine y verifica su estado de salida.

Este patrón es equivalente al uso de tuberías en la *shell*:

```
echo "Hola mundo desde una tubería" | wc -w
```

C.5. Redirección bidireccional

Para comunicación bidireccional entre padre e hijo, se necesitan **dos tuberías**: una para enviar datos del padre al hijo, y otra para enviar datos del hijo al padre.

```
int pipe_to_child[2];
int pipe_from_child[2];

int result1 = pipe(pipe_to_child);
if (result1 == -1)
{
    // Error al crear la tubería...
}

int result2 = pipe(pipe_from_child);
if (result2 == -1)
{
    // Error al crear la tubería...
}

pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo...

    close(pipe_to_child[1]);
    close(pipe_from_child[0]);

    // Leer del padre
    std::array<char, 100> buffer;
    ssize_t bytes_read = read(pipe_to_child[0], buffer.data(), buffer.size());
    if (bytes_read == -1)
    {
        // Error al leer de la tubería...
    }

    buffer[bytes_read] = '\0';
    std::println("El hijo recibió: {}", buffer.data());

    // Enviar respuesta al padre
    std::string_view respuesta = "Respuesta del hijo";
    ssize_t bytes_written = write(pipe_from_child[1], respuesta.data(), respuesta.size());
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
    }

    close(pipe_to_child[0]);
    close(pipe_from_child[1]);
}
```

```

    std::exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // El proceso padre...

    close(pipe_to_child[0]);
    close(pipe_from_child[1]);

    // Enviar mensaje al hijo
    std::string_view mensaje = "Mensaje del padre";
    ssize_t bytes_written = write(pipe_to_child[1], mensaje.data(), mensaje.size()); ④
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
    }

    close(pipe_to_child[1]);

    // Leer respuesta del hijo
    std::array<char, 100> buffer;
    ssize_t bytes_read = read(pipe_from_child[0], buffer.data(), buffer.size()); ⑥
    if (bytes_read == -1)
    {
        // Error al leer de la tubería...
    }

    buffer[bytes_read] = '\0';
    std::println("Padre recibió: {}", buffer.data());

    close(pipe_from_child[0]);
    wait(nullptr);
}
else
{
    // Error al crear el proceso hijo...
}

```

- ① El hijo cierra el extremo de escritura de `pipe_to_child` y el extremo de lectura de `pipe_from_child`. Solo se dejan abiertos los extremos que va a usar.
- ② El padre cierra el extremo de lectura de `pipe_to_child` y el extremo de escritura de `pipe_from_child`.
- ③ El hijo espera para leer desde `pipe_to_child[0]` los datos enviados por el padre desde `pipe_to_child[1]`.
- ④ El padre escribe datos en el extremo de escritura de `pipe_to_child[1]` y cierra ese

extremo para enviar EOF al hijo.

- ⑤ Cuando el hijo recibe los datos, escribe una respuesta en `pipe_from_child[1]` para que el padre la lea.
- ⑥ El padre lee la respuesta del hijo desde `pipe_from_child[0]`.

Este patrón permite una comunicación completa entre dos procesos, donde ambos pueden enviar y recibir datos.